

Using Middleware in ASP.NET Core Web APIs

Understanding Middlewares in .NET

A request passes through one checkpoint at a time, each doing a single job, before boarding.



Middleware is one of the core functionalities of any ASP.NET Core Web Application with respect to ASP.NET Core Web API Request Processing Life Cycle. When we send a request from the client to the server, middleware comes into the picture. A Middleware is a software component that sits between the incoming HTTP request and your application. It can

- Inspect the request
- Modify the request
- Stop the request
- Pass the request to the next middleware
- Inspect the response
- Modify the response

Think of middleware as security checkpoints at an airport as shown in Image.

How HTTP Request Pipeline Works:

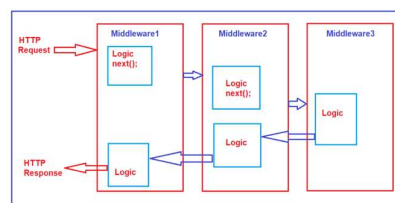
The client i.e. a browser and on the other hand, we have the server where our ASP.NET Core Web API application is hosted. Imagine, the Web API Application has three controllers. So, when the client sends a request to the server, we generally believe that it is the controller action method that is going to serve the request and then we get the response.

But in reality, before hitting the controller action method, the request has to pass through a pipeline. Once the pipeline is completed, then only it navigates the request to the corresponding controller action method.

Middleware in Action:

The Request Pipeline in ASP.NET Core Web API Application can have multiple middleware, which is a piece of code with some logic. Whenever a request comes from a client to the server, then the request comes to the first middleware which is registered in the request pipeline.

The code or logic which is there in Middleware1 will be executed and then if it will call the next method, then the request goes to the next middleware which is registered in the request processing pipeline i.e. Middleware2 and then to MW3 and so on as shown in a line diagram below.



Middleware Examples:

1. **Routing:** If you want to implement Routing in your application, then you need to use Routing Middleware in the HTTP Request Processing pipeline.
2. **Authentication:** If you want to authenticate the user then you need to use Authentication Middleware.
3. **Authorize:** The Authorize Middleware is used to Authorize the users while accessing a specific resource.
4. **Log:** If you want to log request and response while processing, then you need Middleware.
5. **Exception Middleware:** You can also use Middleware to handle the exception globally.

Configure Middleware Components:

With new versions of ASP.NET Core this is configured in Main() method of Program.cs, In earlier versions it was configured within the Configure method of the Startup class. The Startup class is the class that is going to run when the application starts.

While configuring or using middleware the ASP.NET Core Middleware components are executed in the same order as they are added to the pipeline. So, it is our key responsibility to take care when adding middleware components. In nutshell **Order Is Important**.

Run, Use, Next, and Map Methods in Middleware:

In order to work with ASP.NET Core Middleware Components, we need to learn about few methods are as follows:

- **Run() Method:** The Run() Extension Method is used to complete the Middleware Execution.
- **Use() Method:** The Use() Extension Method is used to insert a new Middleware component to the Request Processing Pipeline.
- **Next() Method:** The Next() Extension Method is used to call the next middleware component in the request processing pipeline.
- **Map() Method:** The Map() Extension Method is used to map the Middleware to a specific URL.

Let's see some practical approach

Create a default ASP.NET Core Controller based Web API. Add following in Program.cs code block.

```
namespace MiddlewareDemo
{
    public class Program
    {
        public static void Main(string[] args)
        {
            var builder = WebApplication.CreateBuilder(args);
            // Add services to the container.
            builder.Services.AddControllers();
            // Learn more about configuring Swagger/OpenAPI at
            // https://aka.ms/aspnetcore/swashbuckle
            builder.Services.AddEndpointsApiExplorer();
            builder.Services.AddSwaggerGen();
            var app = builder.Build();
        }
    }
}
```

Free, Open-Source, Cross-Platform Framework

```
// Configure the HTTP request pipeline.
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}
/* trying to use middleware to log request and response data */
app.Use(async (context, next) =>
{
    Console.WriteLine("-----");
    Console.WriteLine("Middleware 1 - Before");

    await next();

    Console.WriteLine("Middleware 1 - After");
    Console.WriteLine("-----");
});

app.Use(async (context, next) =>
{
    Console.WriteLine("Middleware 2 - Before");

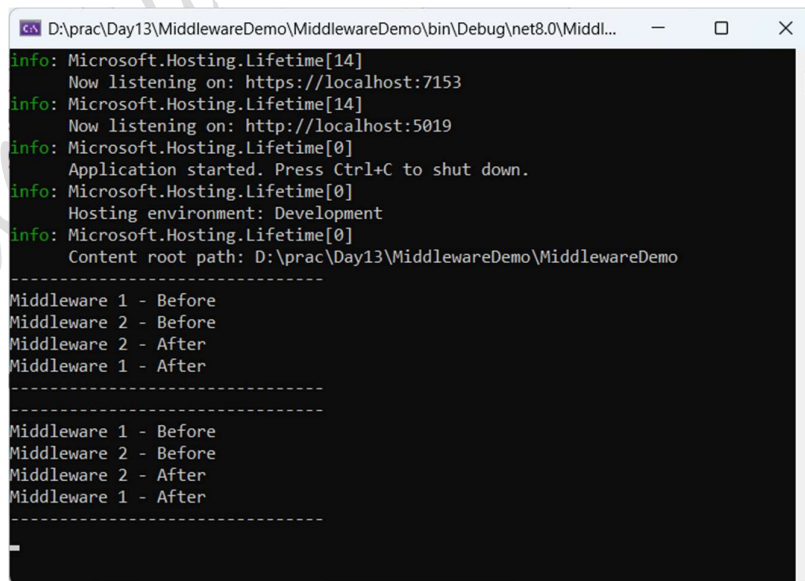
    await next();

    Console.WriteLine("Middleware 2 - After");
});
app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();
app.Run();
}
```

Run application and use following URL:

<https://localhost:7153/weatherforecast>

Observe your console output:



```
D:\prac\Day13\MiddlewareDemo\MiddlewareDemo\bin\Debug\net8.0\Middl...
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: https://localhost:7153
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5019
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: D:\prac\Day13\MiddlewareDemo\MiddlewareDemo
-----
Middleware 1 - Before
Middleware 2 - Before
Middleware 2 - After
Middleware 1 - After
-----
Middleware 1 - Before
Middleware 2 - Before
Middleware 2 - After
Middleware 1 - After
-----
```

Creating Your Own Middleware

While working with the real-time applications in ASP.NET Core Web API, it is a common requirement to create **Custom Middleware** Components. ASP.NET Core provides a rich set of built-in middleware components, but in some scenarios, you might want to write a custom middleware. Suppose you want to:

- Logging all requests and responses
- Global exception handling
- Request/response timing
- API key validation
- Correlation IDs for tracing
- Adding security headers
- Multi-tenant request resolution
- Custom request/response auditing

Custom middleware is commonly used in scenarios listed above.

Let's add one class `MyCustomMW.cs` to your project. Better to add it in a separate folder named `Middlewares`. In order to make a class a Middleware component, the class needs to be inherited from the `IMiddleware` interface. `IMiddleware` interface belongs to `Microsoft.AspNetCore.Http` namespace. We need to implement the `InvokeAsync` method, write your logic within the `InvokeAsync` method as shown below:

```
public class MyCustomMW : IMiddleware
{
    public async Task InvokeAsync(HttpContext context, RequestDelegate next)
    {
        await context.Response.WriteAsync("Custom Middleware MyCustomMW's  
Incoming Request \n");
        await next(context);
        await context.Response.WriteAsync("Custom Middleware MyCustomMW's  
Outgoing Response \n");
    }
}
```

If you want to use any custom service, before using it, you must inject the service to the built-in dependency injection container. You can inject the service in `Program.cs` using following statement:

```
Builder.Services.AddTransient<Middlewares.MyCustomMW>();
```

Once done you need to register the Custom Middleware in the HTTP Request Processing Pipeline using following statement in `Program.cs` after building app.

```
app.UseMiddleware<Middlewares.MyCustomMW>();
```

Now run your application you can observe you execution in browser.

